Tradução para Português do Brasil (parcial)

Arquivo original: TG2_Bruno_Silveira.pdf

Conteúdo traduzido: Agradecimentos, Abstract (tradução) e Capítulo 1 (Introdução).

AGRADECIMENTOS

Em primeiro lugar, agradeço ao meu orientador, Prof. Dr. Gabriel Nazar, que me guiou ao longo deste projeto e também me incentivou a não utilizar quaisquer LLMs na redação desta dissertação. Submeter este trabalho sabendo que cada palavra foi escrita por mim mesmo é extremamente gratificante.

Também gostaria de agradecer à Rheinland-Pfälzische Technische Universität em Kaiserslautern pela oportunidade de estudar lá por um ano, período em que fui introduzido pela primeira vez a esta área de pesquisa. Embora nenhum texto nem código deste projeto tenha sido desenvolvido lá, este trabalho provavelmente não existiria se não fosse o tempo em que atuei como assistente de pesquisa sob a supervisão do Dr. Vladimir Rybalkin. Além disso, agradeço ao Dr. Kai Lampka, da RPTU, que gentilmente me forneceu uma placa de desenvolvimento da NXP como seu aluno, a qual foi utilizada neste projeto.

Por fim, dedico estes agradecimentos aos meus pais e à minha namorada, cujo apoio foi importante durante este último semestre.

RESUMO (tradução do ABSTRACT)

O uso de Redes Neurais Artificiais (do inglês, ANNs) em aplicações com sensores embarcados, como reconhecimento de imagens e detecção de anomalias em sinais, é difundido, normalmente envolvendo o envio de dados por meio de uma rede para que uma máquina mais potente realize o processamento pesado. TinyML é um campo de pesquisa que propõe executar uma versão compactada da rede neural diretamente no processador embarcado, eliminando a latência da rede e reduzindo o consumo total de energia. Um dos maiores desafios do TinyML é a exploração do espaço de projeto, que pode ser acelerada por meio da Busca de Arquiteturas Neurais (do inglês, NAS). O problema dessa abordagem no contexto do TinyML é que os resultados de desempenho de latência frequentemente são estimados por proxies em vez de serem medidos no dispositivo embarcado alvo, o que pode levar a resultados subótimos. Este trabalho propõe a implementação de uma ferramenta de benchmarking que permite a um servidor NAS compilar, fazer deploy e testar automaticamente a inferência de um modelo TensorFlow Lite proposto em hardware real de um microcontrolador ARM Cortex-M. Essa ferramenta foi então utilizada para executar experimentos em um total de 397 modelos representativos de TinyML para avaliar a qualidade de proxies de estimativa de latência, bem como para documentar o tempo total do processo. Os resultados mostram correlações de 0,88 a 0,99 para o número de MACs e de 0,75 a 0,99 para inferência em CPU da estação de trabalho, além de demonstrar que é possível realizar profiling no MCU na ordem de 20 segundos por modelo.

Palavras-chave: TinyML. automação. benchmarking. UFRGS.

1 INTRODUÇÃO

A Internet das Coisas (IoT, do inglês Internet of Things) é uma tendência atual de desenvolvimentos tecnológicos na qual cada vez mais dispositivos estão conectados e compartilhando dados entre si. Essas aplicações prometem facilitar a vida das pessoas ao concretizar conceitos como “Casas Inteligentes”, além de estarem presentes em áreas importantes como indústria, agricultura e saúde (MOUHA et al., 2021). Nesse cenário, também surgem perspectivas econômicas crescentes, pois projeta-se que o mercado global de IoT alcance valor superior a 8 trilhões de dólares até 2030 (AL-SARAWI et al., 2020).

No contexto da IoT, sempre que uma ANN (Artificial Neural Network) precisa ser processada com dados coletados por sensores, isso tipicamente é descarregado pela rede para uma máquina computacional mais poderosa, como um computador equipado com GPU (Graphics Processing Unit). Assim, os dispositivos que controlam os sensores normalmente servem apenas para enviar e receber informações, criando uma dependência da rede para tudo o que é relacionado a ML (Machine Learning) (DUTTA; BHARALI, 2021). Isso motivou o surgimento do TinyML, que é o conceito de executar modelos de ML diretamente nesses dispositivos pequenos que ficam na borda da rede, representando um campo de pesquisa promissor no mundo em constante crescimento da Inteligência Artificial (AI) (ABADADE et al., 2023). Um exemplo comum que evidencia a importância do TinyML são aplicações de keyword spotting, que detectam palavras de ativação como “OK Google” (CHEN; PARADA; HEIGOLD, 2014). Para esses serviços, é importante que a detecção da palavra-chave seja processada localmente no dispositivo móvel e, somente após a palavra de ativação ser detectada, o áudio passe a ser transmitido para um servidor.

Executar inferência de ML em dispositivos de borda em vez de um servidor centralizado pode trazer diversas vantagens em alguns casos de uso. Em termos de consumo de energia, a energia associada ao cálculo de inferência no nó pequeno de borda pode ser muito menor do que a energia usada para transmitir os dados de entrada sem fio (BANBURY et al., 2021). De forma semelhante, executar uma versão compactada de um modelo de ML em um pequeno MCU (Microcontroller Unit) em vez de uma GPU consome energia na ordem de 1000 vezes menor (SAHA; SANDHA; SRIVASTAVA, 2022).

Sistemas TinyML também têm o benefício de independência de conectividade de rede, já que toda a computação necessária é executada localmente. Isso é um fator importante em aplicações como wearables de monitoramento de saúde, que poderiam falhar ao detectar uma emergência caso o usuário esteja em um local remoto sem internet. Por exemplo, os autores em (FARAONE; DELGADO-GONZALO, 2020) desenvolveram e implantaram um modelo que detecta arritmia cardíaca a partir de sinais de eletrocardiograma (ECG) e que roda em um microcontrolador de baixo consumo com acurácia adequada. Ser independente da internet também significa que a latência de enviar e receber os dados de entrada pela rede é evitada, o que pode ser mais adequado para sistemas em tempo real (SAHA; SANDHA; SRIVASTAVA, 2022). Para ilustrar isso, podemos comparar faixas usuais de latência de computação em nuvem e de TinyML, que são, respectivamente, 10–500 ms e 0,18–300 ms (ABADADE et al., 2023). Por fim, preocupações com segurança e privacidade também são reduzidas em sistemas TinyML. Em uma configuração típica de computação em nuvem, o servidor conectado à internet pode ser vulnerável a ataques que exporiam dados sensíveis; já com TinyML, os dados nunca deixam o nó de borda e, assim, ficam mais protegidos contra atacantes (SAHA; SANDHA; SRIVASTAVA, 2022).

No que diz respeito ao dispositivo no qual os pequenos modelos de ML irão rodar, o foco padrão de pesquisa era em telefones móveis, como em (TAN et al., 2019), mas recentemente houve uma mudança para incluir também microcontroladores de baixo consumo (BANBURY et al., 2021). Em comparação com CPUs (Central Processing Units) de dispositivos móveis, MCUs consomem, em média, 40 vezes

menos energia (SAHA; SANDHA; SRIVASTAVA, 2022) e são muito baratos, custando na ordem de um dólar (SAHA; SANDHA; SRIVASTAVA, 2022). Outra vantagem de MCUs, em contraste com, por exemplo, FPGAs (Field Programmable Gate Arrays), é que eles são facilmente programáveis e reconfiguráveis (PEREGO et al., 2022). Isso se reflete na revisão feita em (HAN; SIEBERT, 2022), em que foram contabilizados 21 artigos de TinyML que usaram microcontroladores, dos quais a maioria utilizou especificamente microcontroladores ARM Cortex-M, e apenas 2 que usaram FPGAs. Além disso, FPGAs também são significativamente mais caros do que MCUs, o que significa que estes últimos podem ser mais apropriados em projetos sensíveis a custo (GHANATHE, 2025).

Para implantar uma rede neural em um microcontrolador, os pesos do modelo precisam caber na ROM (Read-Only Memory) — normalmente Flash — juntamente com o código que implementa as operações; e os resultados intermediários dos cálculos entre camadas precisam caber na RAM (Random Access Memory) (LIBERIS; DUDZIAK; LANE, 2021). Assim, é desafiador portar modelos comuns de ML para caber em MCUs, pois um microcontrolador típico tem por volta de 1 MB de ROM e 128 kB de RAM (SAHA; SANDHA; SRIVASTAVA, 2022), enquanto os dispositivos mais poderosos alvos desses modelos possuem gigabytes de ROM e RAM (BURRELLO et al., 2021).

Devido à dificuldade de executar modelos existentes em MCUs, torna-se necessário criar novas arquiteturas de modelos que sejam implantáveis nesses ambientes restritos (PEREGO et al., 2022). Construir esses modelos manualmente é uma possibilidade, mas, por causa do tempo necessário, isso frequentemente é substituído por Neural Architecture Search (NAS), uma estratégia evolutiva que automatiza o processo de exploração do espaço de projeto, propondo modelos iterativamente e testando seu desempenho (SAHA; SANDHA; SRIVASTAVA, 2022). O problema dessas abordagens de NAS é que elas normalmente não medem o tempo de inferência no dispositivo (on-device), mas em vez disso usam heurísticas como o número de operações Multiply and Accumulate (MACs), como em (LIBERIS; DUDZIAK; LANE, 2021) e (BURRELLO et al., 2023), ou outros modelos preditivos auxiliares de ML (PEREGO et al., 2022), alegando que medir no dispositivo seria demorado demais. Contudo, é mostrado brevemente em (TAN et al., 2019) que essas métricas podem ser imprecisas e que a estimativa mais precisa de latência de inferência é sempre executar e medir no dispositivo.

Outro aspecto que torna o desenvolvimento de aplicações TinyML mais difícil é a heterogeneidade presente não apenas nos tipos de modelos e camadas disponíveis para implantação, mas também no hardware e no software que podem ser usados. Frameworks de TinyML, que são conjuntos de ferramentas e bibliotecas de software que auxiliam na implantação de modelos em dispositivos pequenos, frequentemente suportam apenas um subconjunto pequeno de arquiteturas de CPU, dispositivos e operadores de ML, o que complica seu uso transparente em pipelines de desenvolvimento automatizados (ABADADE et al., 2023). Como exemplo, mesmo o framework mais popular (HAN; SIEBERT, 2022) e mais agnóstico a plataforma, o TensorFlow Lite for Microcontrollers, ainda exige alguns ajustes manuais de parâmetros e algumas linhas de código customizado para funcionar de forma ótima (DAVID et al., 2021).

Diante disso, este trabalho propõe o desenvolvimento do B.A.L.A.S, uma ferramenta extensível que ajudará no profiling de ANNs em MCUs específicos. A ferramenta fornece argumentos de linha de comando fáceis de usar para, dado um modelo quantizado, gerar um projeto C++ otimizado que o implemente com TensorFlow Lite for Microcontrollers, compilar e fazer deploy do projeto na placa-alvo e medir a latência de inferência no dispositivo. O objetivo da ferramenta é oferecer uma API simples para um usuário que deseje usar medições mais precisas do mundo real como feedback para uma Busca de Arquiteturas Neurais, ou acelerar o processo de profiling para um usuário que está construindo um modelo

manualmente.

Além disso, este trabalho também propõe o uso do B.A.L.A.S para executar experimentos com o objetivo de comparar heurísticas de estimativa de latência com medições reais no dispositivo, de modo a justificar — ou não — o tempo extra usado pelas medições reais. Por fim, este trabalho também busca preencher uma lacuna de pesquisa e documentar quanto tempo todo o processo do B.A.L.A.S leva, para servir como referência e auxiliar outros pesquisadores de TinyML na decisão sobre se o profiling de latência no dispositivo leva tempo demais ou se vale o investimento.